



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Mercy Food – Sprint 1”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco, Alberto Luna Rufino, Eduardo
Loredo García y Jorge Armando López Morales

Grupo: S-204

15/oct/2025

1. Introducción

Mercy Food es una plataforma web diseñada para crear un ecosistema de delivery de comida que conecta a tres tipos de usuarios: clientes, restaurantes locales y repartidores independientes. El objetivo del proyecto es fortalecer la economía local a través de una aplicación moderna, segura y fácil de usar, que gestiona el proceso desde el registro de usuarios hasta la entrega de pedidos.

En su fase actual, el proyecto cuenta con una base sólida que incluye un flujo de autenticación completo, interfaces de usuario personalizadas y un backend funcional preparado para escalar.

2. Características principales

La versión actual de la plataforma incluye las siguientes funcionalidades:

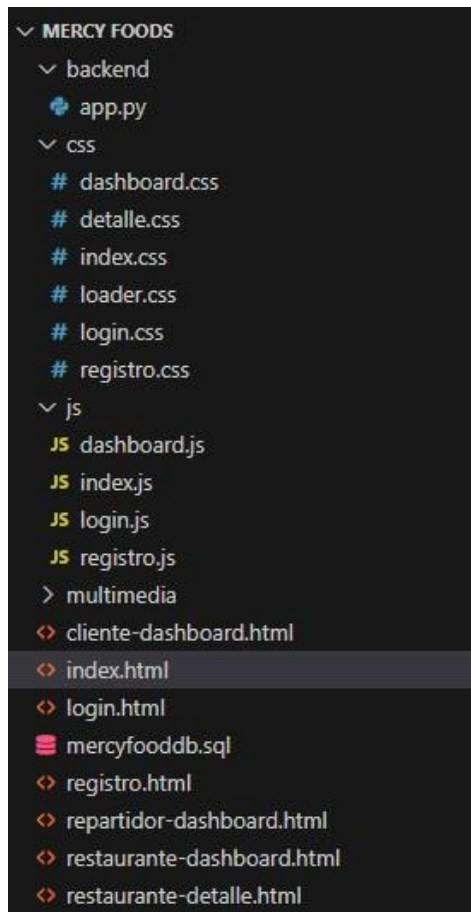
- **Sistema de autenticación por roles:**
 - Registro tripartito: Un único formulario de registro permite a los usuarios crear una cuenta como Cliente, Restaurante o Repartidor, capturando la información específica para cada rol.
 - Login Seguro: Un formulario de inicio de sesión que valida las credenciales contra la base de datos, utilizando contraseñas encriptadas para mayor seguridad.
 - Validación en tiempo real: Los formularios de registro y login ofrecen retroalimentación instantánea al usuario, validando campos como el formato del correo y la fortaleza de la contraseña.
- **Dashboards personalizados por rol:**
 - Tras un inicio de sesión exitoso, cada usuario es redirigido a un dashboard específico diseñado para sus necesidades.
 - Dashboard de cliente: Interfaz para la gestión de pedidos y perfil.
 - Dashboard de restaurante: Panel de control para la gestión de menú y pedidos entrantes.
 - Dashboard de repartidor: Interfaz para visualizar y gestionar entregas disponibles.

- **Experiencia de usuario mejorada (UI/UX):**

- Interfaces modernas y responsivas: Todos los dashboards y formularios están diseñados para adaptarse a diferentes tamaños de pantalla, desde escritorio hasta móviles.
- Notificaciones Modales: Se han reemplazado las alertas nativas del navegador por modales estilizados para comunicar mensajes de éxito o error, mejorando la estética y la experiencia del usuario.
- Logout Seguro: Se implementó un modal de confirmación para el cierre de sesión, previniendo acciones accidentales.
- Loader de Carga: Todas las páginas cuentan con una animación de carga inicial para mejorar la percepción de rendimiento.

3. Estructura del proyecto

El proyecto está organizado con una separación clara entre el backend, el frontend y los recursos multimedia.



4. Tecnologías utilizadas

- **Frontend:**

- HTML5
- CSS3 (con Flexbox y Grid para responsividad)
- JavaScript: Para la interactividad del cliente, validaciones y llamadas a la API (fetch).
- Bootstrap para agilizar el proceso de responsividad

- **Backend:**

- Python 3
- Flask: Micro-framework para el servidor web y la API REST.
- Flask-Bcrypt: Para la encriptación y verificación de contraseñas.
- Flask-CORS: Para permitir las solicitudes desde el frontend.

- **Base de Datos:**

- Microsoft SQL Server
- pyodbc: Driver para la conexión entre Python y SQL Server.

5. Configuración

Para ejecutar el proyecto en un entorno de desarrollo local, sigue estos pasos:

1. Configurar la base de datos:

- Asegúrate de tener SQL Server instalado y en ejecución.
- Abre un gestor de bases de datos (como SQL Server Management Studio).
- Ejecuta el script `mercyfooddb.sql` para crear la base de datos `MercyFoodDB` y la tabla `Users`.

2. Configurar el Backend:

- Navega a la carpeta del proyecto.

3. Instala las dependencias de Python:

```
pip install Flask Flask-Bcrypt Flask-CORS pyodbc
```

4. Ejecutar la aplicación:

cd backend py

app.py

El servidor se iniciará en <http://127.0.0.1:5000>

5. Acceder al frontend

Abre tu navegador web y ve a <http://127.0.0.1:5000> para ver la página de inicio. Las demás páginas (login, registro) son accesibles desde allí.

6. Endpoints de la API

La API REST gestiona la lógica de negocio y la interacción con la base de datos.

- POST /api/register
 - **Descripción:** Registra un nuevo usuario en la base de datos.
 - **Cuerpo (Body):** Un objeto JSON con email, password, nombre, role y campos adicionales según el rol.
 - **Respuesta exitosa (201):** { "message": "¡Cuenta creada con éxito!" }
 - **Respuesta de error (409):** { "message": "El correo electrónico ya está registrado." }
- POST /api/login
 - **Descripción:** Autentica a un usuario existente.
 - **Cuerpo (Body):** Un objeto JSON con email y password.
 - **Respuesta exitosa (200):** { "message": "Inicio de sesión exitoso.", "user": { ... } }
 - **Respuesta de error (401):** { "message": "Credenciales incorrectas..." }
- GET /api/faq
 - **Descripción:** Obtiene la lista de preguntas frecuentes desde la base de datos.
 - **Respuesta exitosa (200):** { "faqs": [{ "id": 1, "question": "...", "answer": "..." }] }